os lab1

林家立 241220068

2961184218@qq.com

一.实验进程

完成实验要求的所有内容

二.实验结果

三.实验修改代码位置

1./bootloader/start.asm:

4处todo

1. 关闭中断。

   ```
   cli
   ```

2. 启用 A20 地址线。

   ```
   in al, 0x92
   or al, 0x02
   out 0x92, al
   ```

3. 开启保护模式。

将cr0最低位设为0即可

mov eax,cr0
or al,0x01
mov cr0, eax

4. 跳转到 `bootMain`。

jmp bootMain

2./bootloader/boot.c

```
void bootMain(void) {
    Implement with Codex
    /* TODO: 循环读取扇区
     */
    for (int i = 1; i <= SECTOR_COUNT; i++) {
        readSect((void *)APP_START + (i-1) * 512, i);
    }
    ((void (*)(void))APP_START)();

}
```

通过阅读bootloader/io.asm理解到readSect用法后即可写出

3./app/io.asm

```
readRTC:
    ; TODO: 实现readRTC，函数定义见main.h

; 等待 RTC 不在更新中（寄存器 0x0A 的 bit7 变 0）
wait_uip:
    mov al, 0x8a
    out 0x70, al          ; 选寄存器 0x0A
    in  al, 0x71
    test al, 0x80
    jnz wait_uip

    ;禁用NMI
    mov al, [esp+4]
    or  al, 0x80
    out 0x70,al

    ;读值
    in al,0x71
    mov bl,al

    ;重新启用nmi
    mov  al, 0x00
    out  0x70, al

    mov al,bl
    ret
```

根据实验课ppt提示，在禁用mni情况下检查uip（mov al，0x8a），直到uip结束，然后在禁用mni情况下将参数写到0x70后在0x71读值，最后重新启用nmi

4.app/main.c

```
int bcdToStr(char bcd, char *buf) {
    Implement with Codex
    // TODO: 把bcd转换为string并存入buf中
    int bit1 = (bcd >> 4) & 0x0F;  // 高 4 位
    int bit0 = bcd & 0x0F;         // 低 4 位

    buf[0] = '0' + bit1;
    buf[1] = '0' + bit0;
    buf[2] = '\0';
    return 2;
}
```

```c
int main(void) {
    char rtc_time_str[8] = "00:00:00";
    int row = 12;
    int col = (80 - 8) / 2;
    char hint_str[] = "Current Time:";
    displayStr(row - 1, (80 - 12) / 2, hint_str, 12);
    while (1) {
        Implement with Codex
        // TODO: 实现展示时间的功能
        char seconds_bcd = readRTC(0x00);
        char minutes_bcd = readRTC(0x02);
        char hours_bcd = readRTC(0x04);

        bcdToStr(hours_bcd, rtc_time_str);
        rtc_time_str[2] = ':';
        bcdToStr(minutes_bcd, rtc_time_str + 3);
        rtc_time_str[5] = ':';
        bcdToStr(seconds_bcd, rtc_time_str + 6);
        displayStr(row, col, rtc_time_str, 8);
        delay();
    }
    return 0;
}
```

bcdToStr：通过看实验课ppt，将bcd分成高低各4位，与0xF与后即可得高低4位的值，然后再将其放进buf中即可

展示时间：将时，分，秒分别赋值即可

四.遇到的问题

修改readRTC时，一开始发现时钟始终是00：00：00，后面发现是重新启用mni时我通过al传值给0x70，然后没有复原al。

五.思考题

1.未使用debug/log

2.bootloader和app代码的栈初始位置是否一致，是否可以不一致？设置的原则是什么？

一致：

2.1不一致

在 `bootloader/start.asm` 中定义了栈顶地址：

```
%define BOOT_STACK_TOP 0x10000
```

```
mov eax, BOOT_STACK_TOP
mov esp, eax
```

app 的 `start.asm` 中没有显式设置栈指针

但app 被加载到地址 `0x10000`（在 `bootloader/boot.c` 中定义）链接地址也设置为 `0x10000`（在 `app/Makefile` 中 `-Ttext 0x10000`）

所以在运行时，bootloader和app的栈空间一致

2.2可以不一致

2.3原则

1. **内存空间分离**：不同程序的栈应该设置在不同的内存区域，避免冲突
2. **栈增长方向**：x86 架构中栈向下增长（从高地址到低地址），所以栈顶应设置在内存区域的高地址部分
3. **内存保护**：栈空间应该足够大，并且不会被其他代码或数据覆盖

3.app的链接时设置的逻辑地址起始位置是否可以与实际加载位置不一致?

不可以。如果链接地址与加载地址不一致，会导致以下问题：

1. **指令跳转错误**：程序中的 `call`、`jmp` 等指令会跳转到基于链接地址计算的位置，而不是实际加载的位置
2. **数据访问错误**：全局变量、常量等数据的访问会使用错误的地址
3. **程序崩溃**：由于地址不匹配，程序会执行到无效内存区域，导致崩溃